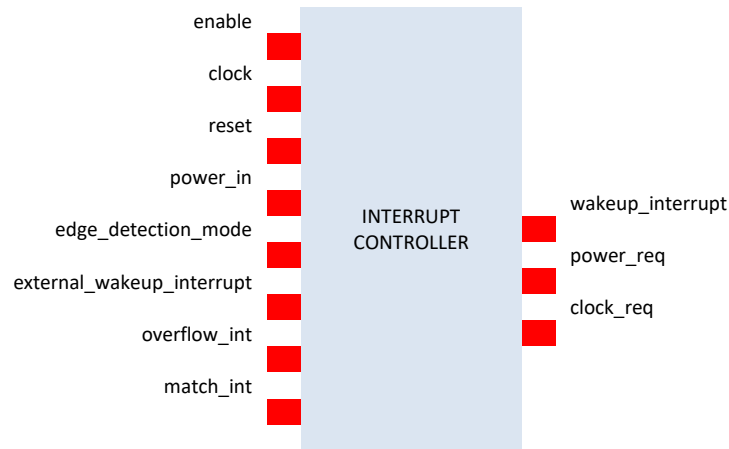


INTERRUPT_CONTROLLER



Block Diagram

1. Overview

The `s2c_interrupt_controller` module is an **edge-configurable interrupt aggregation and wakeup controller**.

It combines multiple interrupt sources and generates a **configurable edge-detected wakeup interrupt** suitable for:

- Low-power wakeup systems
- Timer-based interrupt systems
- External event detection

2. Key Features

2.1 Multiple Interrupt Source Aggregation

The module combines three interrupt sources:

Source	Description
<code>overflow_int</code>	From counter overflow
<code>match_int</code>	From comparator match
<code>external_wakeup_interrupt</code>	External interrupt source

Combined logic:

```
timer_int = overflow_int | match_int | external_wakeup_interrupt;
```

This creates a unified interrupt signal.

2.2 Programmable Edge Detection

Edge detection mode is controlled via:

```
edge_detection_mode [1:0]
```

Mode	Value	Behavior
Disabled	0	No interrupt generated
Rising Edge	1	Trigger on 0 → 1
Falling Edge	10	Trigger on 1 → 0
Both Edges	11	Trigger on any transition

Edge detection is implemented using:

- Previous state storage (`timer_int_d`)
- XOR and logic comparison

This makes the module highly flexible.

2.3 Power-Aware Design

- `power_req = enable`
- `clock_req = power_in & enable`

Interrupt logic operates only when:

- Enabled
- Powered
- Clock active

If any of the following are low:

- `reset`
- `enable`
- `power_in`

Then:

- `wakeup_interrupt = 0`

Ensures safe low-power behavior.

2.4 Synchronous Edge Detection Logic

On every clock edge:

1. Current interrupt state stored.
2. Previous state retained.
3. Edge comparison performed.
4. Wakeup interrupt generated accordingly.

This provides:

- Clean glitch-free detection
 - Deterministic behavior
 - Hardware-safe edge detection
-

2.5 Clean Reset Behavior

On:

- Reset
- Disable
- Power loss

The module:

- Clears `wakeup_interrupt`
- Clears previous state register

Ensures predictable startup.

3. Signal Description

Signal	Direction	Description
enable	Input	Global enable
clock	Input	System clock
reset	Input	Asynchronous reset
power_in	Input	Power availability
edge_detection_mode	Input	Edge configuration control
external_wakeup_interrupt	Input	External interrupt
overflow_int	Input	Counter overflow interrupt
match_int	Input	Comparator match interrupt
wakeup_interrupt	Output	Final wakeup interrupt
power_req	Output	Power request
clock_req	Output	Clock request

4. Functional Flow

1. Combine interrupt sources → timer_int
 2. Store previous value → timer_int_d
 3. Compare current vs previous
 4. Generate wakeup interrupt based on selected edge mode
-